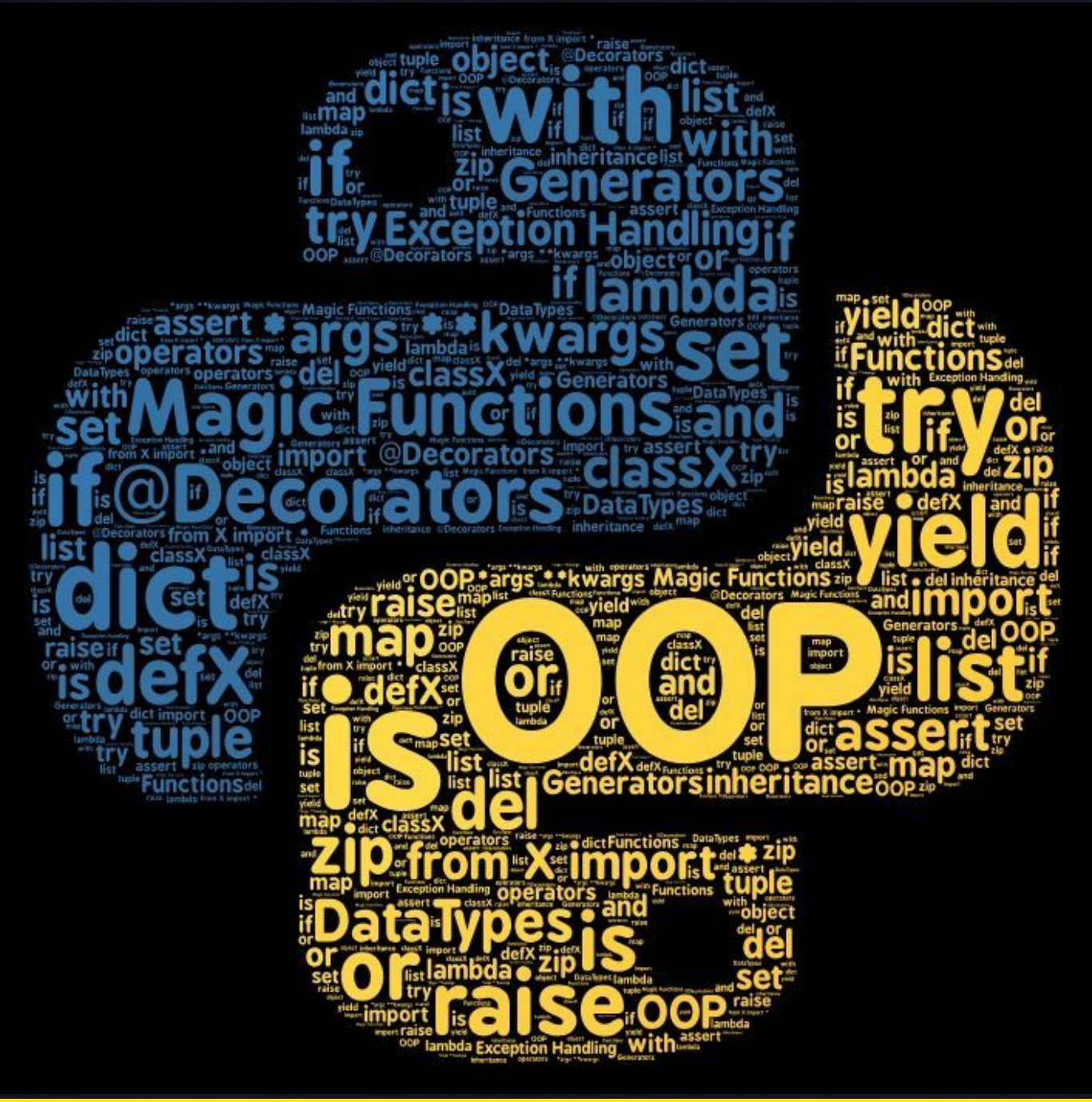


PYTHON PROGRAMMING

Assoc. Prof. Dr. Bora Canbula

Manisa Celal Bayar University
Department of Computer Engineering

https://github.com/canbula/PythonProgramming/ w0p5whf

Python Programming

Instructor

Assoc. Prof. Dr.
Bora CANBULA

Phone

0 (236) 201 21 08

Email

bora.canbula@cbu.edu.tr

Office Location

Dept. of CENG

Office C233

Office Hours

10 am – 12 am, Fridays

Course Overview

Python Programming (Teams Code: w0p5whf)

This course is about basic concepts in Python Programming. Students will learn the data structures, functions, classes, and the special structures of Python. They will also learn how to create basic desktop applications with a GUI, web applications as APIs, basic games as projects. In this course, we will use GitHub actions for assignments.

Required Text

Advanced Guide to Python 3 Programming, Springer, *John Hunt*

Python for Everybody, PY4E, *Charles Severance*

Course Materials

- Python 3.x (Anaconda is preferred)
- Jupyter Notebook from Anaconda
- Pycharm from JetBrains / Microsoft Visual Studio Code
- PC with a Linux distro or a Linux terminal in Windows 10/11.

Course Schedule

Week	Subject	Week	Subject
01	Basic Concepts in Python	08	Desktop Programming
02	Sequences	09	Desktop Programming
03	Functions	10	Web Programming
04	Decision Structures	11	Web Programming
05	Loops	12	Game Programming
06	Classes	13	Game Programming
07	Special Structures	14	Student Projects

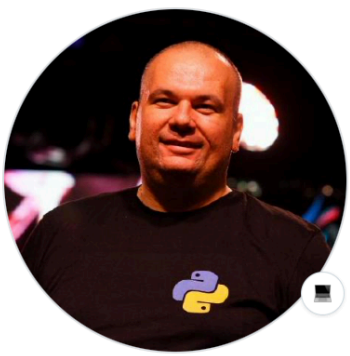
w0p5whf

First Quiz

https://forms.cloud.microsoft/r/jDhD5ER3rC

Repository for the Course

https://github.com/canbula



Bora Canbula
canbula

Assoc. Prof. Dr. **@mcbuceng** studying data science, high performance computing and theoretical nuclear physics.

Pinned

Customize your pins

ParallelProgramming Public
Repository for Parallel Programming course given by Assoc. Prof. Dr. Bora Canbula at Computer Engineering Department of Manisa Celal Bayar University.
Python 78 189

NumericalAnalysis Public
Repository for Numerical Analysis course given by Assoc. Prof. Dr. Bora Canbula at Computer Engineering Department of Manisa Celal Bayar University.
Python 30 68

Statistics Public
Repository for Statistics course given by Assoc. Prof. Dr. Bora Canbula at Department of Computer Engineering, Manisa Celal Bayar University
Python 50 105

PythonProgramming Public
Repository for Python Programming course given by Assoc. Prof. Dr. Bora Canbula at Computer Engineering Department of Manisa Celal Bayar University.
Python 22 85

ieee754 Public
Python module which finds the IEEE-754 representation of a floating point number.
Python 30 6

DataScience Public
Repository for Data Science course given by Assoc. Prof. Dr. Bora Canbula at Computer Engineering Department of Manisa Celal Bayar University.
Jupyter Notebook 6 15

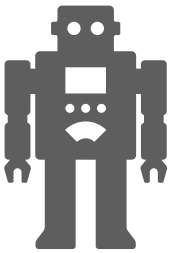
First Homework

Week01/info_firstname_lastname.py

- [x] A string variable with the name student_id that contains your student id.
- [x] A string variable with the name full_name that contains your full name.

Natural Languages vs. Programming Languages

A language is a tool for expressing and recording thoughts.



Computers have their own language called **machine** language. Machine languages are created by humans, no computer is currently capable of creating a new language. A complete set of known commands is called an instruction list (IL).

The difference is that human languages developed naturally. They are still evolving, new words are created every day as old words disappear. These languages are called **natural** languages.



Elements of a Language

- **Alphabet** is a set of symbols to build words of a certain language.
- **Lexis** is a set of words the language offers its users.
- **Syntax** is a set of rules used to determine if a certain string of words forms a valid sentence.
- **Semantics** is a set of rules determining if a certain phrase makes sense.





Machine Language vs. High-Level Language

The IL is the alphabet of a machine language. It's the computer's mother tongue.

High-level programming language enables humans to write their programs and computers to execute the programs. It is much more complex than those offered by ILs.

A program written in a high-level programming language is called a **source code**. Similarly, the file containing the source code is called the **source file**.

Compilation vs. Interpretation

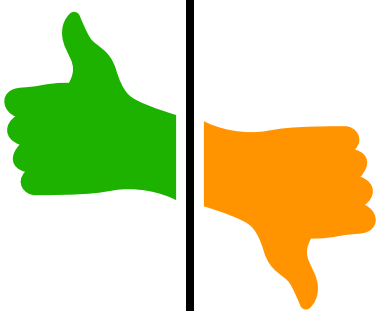
There are two different ways of transforming a program from a high-level programming language into machine language:

Compilation: The source code is translated once by getting a file containing the machine code.

Interpretation: The source code is interpreted every time it is intended to be executed.

Compilation

- The execution of the translated code is usually faster.
- Only the user has to have the compiler. The end user may use the code without it.
- The translated code is stored using machine language. Your code are likely to remain your secret.



- The compilation itself may be a very time-consuming process
- You have to have as many compilers as hardware platforms you want your code to be run on.

Interpretation

- You can run the code as soon as you complete it, there are no additional phases of translation.
- The code is stored using programming language, not machine language. You don't compile your code for each different architecture.

- Don't expect interpretation to ramp up your code to high speed
- Both you and the end user have the interpreter to run your code.

What is Python?

Python is a widely-used, interpreted, object-oriented, and high-level programming language with dynamic semantics, used for general-purpose programming.

Python was created by Guido van Rossum. The name of the Python programming language comes from an old BBC television comedy sketch series called Monty Python's Flying Circus.





Guido van Rossum

Python Goals

- an **easy and intuitive** language just as powerful as those of the major competitors
- **open source**, so anyone can contribute to its development
- code that is as **understandable** as plain English
- **suitable for everyday tasks**, allowing for short development times



Why Python?

- easy to learn
- easy to teach
- easy to use
- easy to understand
- easy to obtain, install and deploy

Why not Python?

- low-level programming
- applications for mobile devices

Python Implementations

An implementation refers to a program or environment, which provides support for the execution of programs written in the Python language.

- **CPython** is the traditional implementation of Python and it's most often called just "Python".
- **Cython** is a solution which translate Python code into "C" to make it run much faster than pure Python.
- **Jython** is an implementation follows only Python 2, not Python 3, written in Java.
- **PyPy** represents a Python environment written in Python-like language named RPython (Restricted Python), which is actually a subset of Python.
- **MicroPython** is an implementation of Python 3 that is optimized to run on microcontrollers.

Start Coding with Python

- **Editor** will support you in writing the code. The Python 3 standard installation contains a very simple application named IDLE (Integrated Development and Learning Environment).
- **Console** is a terminal in which you can launch your code.
- **Debugger** is a tool, which launches your code step-by-step to allow you to inspect it.

first.py

```
print("Python is the best!")
```



https://forms.cloud.microsoft/r/gW

in-class quiz

Function Name
A function can cause some effect
or evaluate a value, or both.

Where do functions come from?
- From Python itself
- From modules
- From your code

first.py

```
print("Python is the best!")
```

Argument
- Positional arguments
- Keyword arguments

print(**objects*, *sep*=' ', *end*='\n', *file*=None, *flush*=False)

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

All non-keyword arguments are converted to strings like `str()` does and written to the stream, separated by *sep* and followed by *end*. Both *sep* and *end* must be strings; they can also be `None`, which means to use the default values. If no *objects* are given, `print()` will just write *end*.

The *file* argument must be an object with a `write(string)` method; if it is not present or `None`, `sys.stdout` will be used. Since printed arguments are converted to text strings, `print()` cannot be used with binary mode file objects. For these, use `file.write(...)` instead.

Output buffering is usually determined by *file*. However, if *flush* is true, the stream is forcibly flushed.

Literals

A literal is data whose values are determined by the literal itself. Literals are used to encode data and put them into code.

literals.py

```
print("7")
print(7)
print(7.0)
print(7j)
print(True)
print(0b10)
print(0o10)
print(0x10)
print(7.4e3)
```

- String
- Integer
- Float
- Complex
- Boolean
- Binary
- Octal
- Hexadecimal
- Scientific Notation

Basic Operators

An operator is a symbol of the programming language, which is able to operate on the values.

Multiplication

```
print(2 * 3)      Integer
print(2 * 3.0)    Float
print(2.0 * 3)    Float
print(2.0 * 3.0)  Float
```

Division

```
print(6 / 3)      Float
print(6 / 3.0)    Float
print(6.0 / 3)    Float
print(6.0 / 3.0)  Float
```

Exponentiation

```
print(2**3)       Integer
print(2**3.0)     Float
print(2.0**3)     Float
print(2.0**3.0)   Float
```

Floor Division

```
print(6 // 3)     Integer
print(6 // 3.0)   Float
print(6.0 // 3)   Float
print(6.0 // 3.0) Float
```

Modulo

```
print(6 % 3)      Integer
print(6 % 3.0)    Float
print(6.0 % 3)    Float
print(6.0 % 3.0)  Float
```

Addition

```
print(-8 + 4)     Integer
print(-4.0 + 8)   Float
```

Operator Priorities

An operator is a symbol of the programming language, which is able to operate on the values.

priorities.py

```
print(9 % 6 % 2)
print(2**2**3)
print(2 * 3 % 5)
print(-3 * 2)
print(-2 * 3)
print(-(2 * 3))
```

- + (unary)
- - (unary)
- ** (right-sided binding)
- *
- /
- //
- % (left-sided binding)
- + (binary)
- - (binary)

Variables

Variables are symbols for memory addresses.

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions

A	E	L	R
abs()	enumerate()	len()	range()
aiter()	eval()	list()	repr()
all()	exec()	locals()	reversed()
anext()			round()
any()	F	M	
ascii()	filter()	map()	S

hex(*x*)

Convert an integer number to a lowercase hexadecimal string prefixed with "0x". If *x* is not a Python `int` object, it has to define an `__index__()` method that returns an integer. Some examples:

```
>>> hex(255)
'0xff'
>>> hex(-42)
'-0x2a'
```

classmethod()	help()	ord()	type()
compile()	hex()		
complex()		P	V
	I	pow()	vars()
D	id()	print()	

id(*object*)

Return the "identity" of an object. This is an integer which is guaranteed to be unique and constant for this object during its lifetime. Two objects with non-overlapping lifetimes may have the same `id()` value.

https://docs.python.org/3/library/functions.html

Identifier Names

For variables, functions, classes etc. we use identifier names. We must obey some rules and we should follow some naming conventions.

- Names are case sensitive.
- Names can be a combination of letters, digits, and underscore.
- Names can only start with a letter or underscore, can not start with a digit.
- Keywords can not be used as a name.

keyword — Testing for Python keywords

Source code: Lib/keyword.py

This module allows a Python program to determine if a string is a keyword or soft keyword.

`keyword.`**`iskeyword`**`(s)`
Return `True` if *s* is a Python keyword.

`keyword.`**`kwlist`**
Sequence containing all the keywords defined for the interpreter. If any keywords are defined to only be active when particular `__future__` statements are in effect, these will be included as well.

`keyword.`**`issoftkeyword`**`(s)`
Return `True` if *s* is a Python soft keyword.

New in version 3.9.

`keyword.`**`softkwlist`**
Sequence containing all the soft keywords defined for the interpreter. If any soft keywords are defined to only be active when particular `__future__` statements are in effect, these will be included as well.

New in version 3.9.



https://forms.cloud.microsoft/r/[illegible]

in-class quiz

Your First Homework

Week01/info_firstname_lastname.py

- ☑ A string variable with the name student_id that contains your student id.
- ☑ A string variable with the name full_name that contains your full name.

Week02/types_firstname_lastname.py

- ☑ An integer with the name: my_int
- ☑ A float with with the name: my_float
- ☑ A boolean with the name: my_bool
- ☑ A complex with the name: my_complex

Rules for your pull requests

- ✓ Please run your code first in your computer, do not submit codes with syntax errors.
- ✓ Submit your code to WeekXX folder. WeekXX/hw is for me to move accepted works.
- ✓ If a change is requested, please edit the existing pull request, don't open a new one.

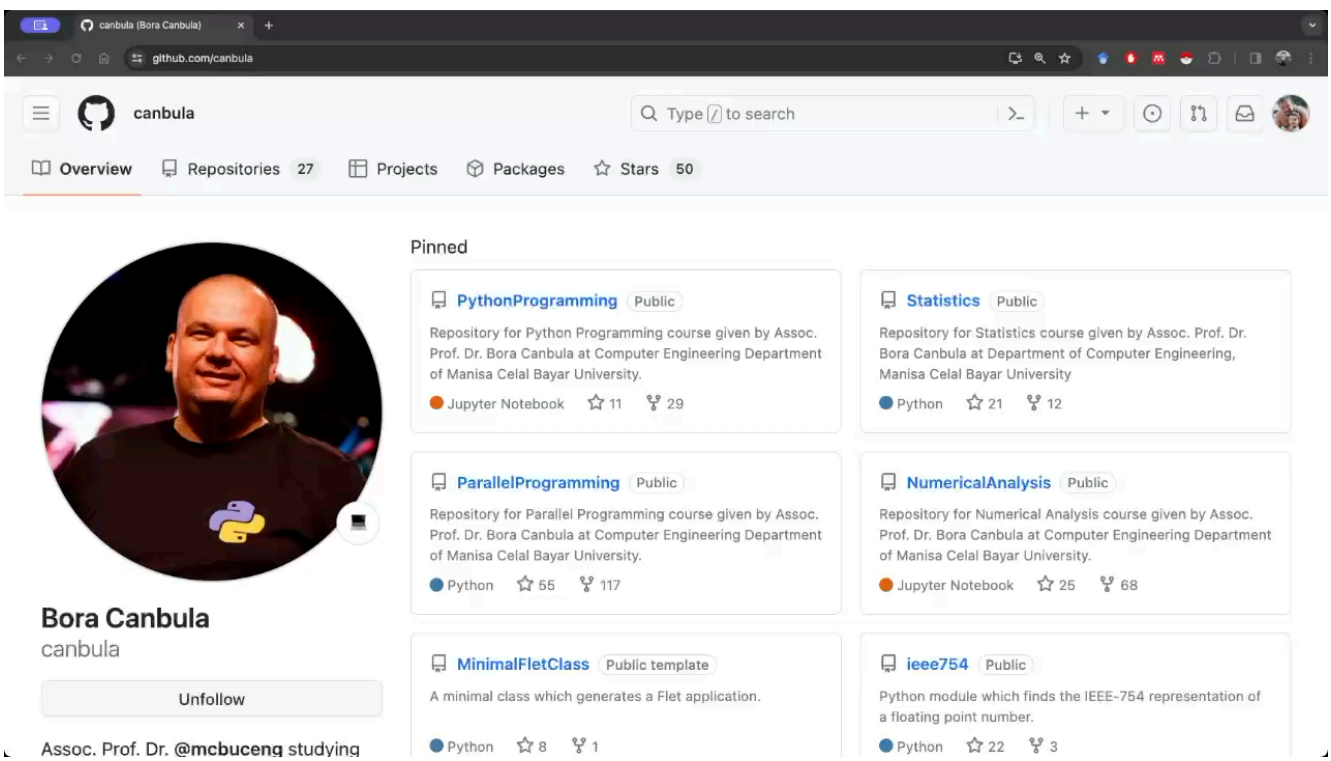
Your First Homework

Week01/info_firstname_lastname.py

- ☑ A string variable with the name student_id that contains your student id.
- ☑ A string variable with the name full_name that contains your full name.

Week02/types_firstname_lastname.py

- ☑ An integer with the name: my_int
- ☑ A float with with the name: my_float
- ☑ A boolean with the name: my_bool
- ☑ A complex with the name: my_complex



Equality & Identity & Comparison

The operators `is` and `is not` test for an object's identity: `x is y` is true if and only if *x* and *y* are the same object. An Object's identity is determined using the `id()` function. `x is not y` yields the inverse truth value. [4]

Equality

```
a = 4
print(a == 4) ✓
print(id(a) == id(4)) ✓
print(a is 4) ✓
print(id(a) is id(4)) ✗
```

Left- or Right-sided?

```
x, y, z = 0, 1, 2
print(x == y == z) ✗
print(x == (y == z)) ✓
print((x == y) == z) ✗
```

Inequality

```
print(1 != 2) ✓
print(not 1 == 2) ✓
```

Updated Priority Table

Operator	Type
+, -	unary
**	binary
*, /, //, %	binary
+, -	binary
<, <=, >, >=	binary
!=, ==	binary

Comparison

```
print(1 < 2) ✓
print(1 <= 2) ✓
print(1 > 2) ✗
print(1 >= 2) ✗
```

Chaining

```
print(1 < 2 < 3) ✓
print(1 < 2 > 3) ✗
print(1 < 2 >= 3) ✗
print(1 < 2 <= 3) ✓
```

QUESTION

Using one of the comparison operators in Python, write a simple two-line program that takes the parameter **n** as input, which is an integer, prints **False** if **n** is less than **100**, and **True** if **n** is greater than or equal to **100**.

```
n = int(input())
print(n >= 100)
```

Conditional Execution

if statement

```
if n >= 100:
    print("The number is greater than or equal to 100.")
elif n < 0:
    print("The number is negative.")
else:
    print("The number is less than 100.")
```

Ternary Operator

```
msg = "The number is greater than or equal to 100." if n >= 100 else "The number is less than 100."
print(msg)
```

Identifier Names

For variables, functions, classes etc. we use identifier names. We must obey some rules and we should follow some naming conventions.

Naming Conventions from PEP 8

- **Names to Avoid**
  Never use the characters 'l' (lowercase letter el), 'O' (uppercase letter oh), or 'I' (uppercase letter eye) as single character variable names.
- **Packages**
  Short, all-lowercase names without underscores
- **Modules**
  Short, all-lowercase names, can have underscores
- **Classes**
  CapWords (upper camel case) convention
- **Functions**
  snake_case convention
- **Variables**
  snake_case convention
- **Constants**
  ALL_UPPERCASE, words separated by underscores

Underscore Usage in Identifier Names

- **_single_leading_underscore**
  Weak "internal use" indicator.
  from M import * does not import objects whose names start with an underscore.
- **single_trailing_underscore_**
  Used by convention to avoid conflicts with keyword.
- **__double_leading_underscore**
  When naming a class attribute, invokes name mangling (inside class FooBar, __boo becomes _FooBar__boo)
- **__double_leading_and_trailing_underscore__**
  "magic" objects or attributes that live in user-controlled namespaces (__init__, __import__, etc.). Never invent such names; only use them as documented.

Loops

QUESTION

- The program generates a random number between 1 and 10.
- The user is asked to guess the number.
- The user is given feedback if the guess is too low or too high.
- The user is asked to guess again until the correct number is guessed.

```
r = random.randint(1, 10)
answer = False
while not answer:
    n = int(input("Enter a number: "))
    if n == r:
        print("You guessed it right!")
        answer = True
    elif n < r:
        print("Try a higher number.")
    else:
        print("Try a lower number.")
```

QUESTION

- The user is asked to enter a number.
- The program prints the numbers from 0 to n-1.

```
n = int(input("Enter a number: "))
for i in range(n):
    print(i, end=" ")
```

break and continue

```
n = int(input("Enter a number: "))
for i in range(10):
    if i < n:
        print("The number is not found:", i)
        continue
    if i == n:
        print("The number is found:", i)
        break
```



```
r = random.randint(1, 10)
while True:
    n = int(input("Enter a number: "))
    if n == r:
        print("You guessed it right!")
        break
    elif n < r:
        print("Try a higher number.")
    else:
        print("Try a lower number.")
```

class **range**(*stop*)
class **range**(*start*, *stop*[, *step*])

The arguments to the range constructor must be integers (either built-in `int` or any object that implements the `__index__()` special method). If the *step* argument is omitted, it defaults to `1`. If the *start* argument is omitted, it defaults to `0`. If *step* is zero, `ValueError` is raised.

For a positive *step*, the contents of a range `r` are determined by the formula `r[i] = start + step*i` where `i >= 0` and `r[i] < stop`.

For a negative *step*, the contents of the range are still determined by the formula `r[i] = start + step*i`, but the constraints are `i >= 0` and `r[i] > stop`.

start
The value of the *start* parameter (or `0` if the parameter was not supplied)

stop
The value of the *stop* parameter

step
The value of the *step* parameter (or `1` if the parameter was not supplied)

The advantage of the `range` type over a regular `list` or `tuple` is that a `range` object will always take the same (small) amount of memory, no matter the size of the range it represents (as it only stores the `start`, `stop` and `step` values, calculating individual items and subranges as needed).

Can we use while/for with else?

Lists in Python:

Ordered and mutable sequence of values indexed by integers

Initializing

```
a_list = [] ## empty
a_list = list() ## empty
a_list = [3, 4, 5, 6, 7] ## filled
```

Finding the index of an item

```
a_list.index(5) ## 2 (the first occurence)
```

Accessing the items

```
a_list[0] ## 3
a_list[1] ## 4
a_list[-1] ## 7
a_list[-2] ## 6
a_list[2:] ## [5, 6, 7]
a_list[:2] ## [3, 4]
a_list[1:4] ## [4, 5, 6]
a_list[0:4:2] ## [3, 5]
a_list[4:1:-1] ## [7, 6, 5]
```

Adding a new item

```
a_list.append(9) ## [3, 4, 5, 6, 7, 9]
a_list.insert(2, 8) ## [3, 4, 8, 5, 6, 7, 9]
```

Update an item

```
a_list[2] = 1 ## [3, 4, 1, 5, 6, 7, 9]
```

Remove the list or just an item

```
a_list.pop() ## last item
a_list.pop(2) ## with index
del a_list[2] ## with index
a_list.remove(5) ## first occurence of 5
a_list.clear() ## returns an empty list
del a_list ## removes the list completely
```

```
a_list[4:1:-1] ## [7, 6, 5]
```

Adding a new item

```
a_list.append(9) ## [3, 4, 5, 6, 7, 9]
a_list.insert(2, 8) ## [3, 4, 8, 5, 6, 7, 9]
```

Update an item

```
a_list[2] = 1 ## [3, 4, 1, 5, 6, 7, 9]
```

Remove the list or just an item

```
a_list.pop() ## last item
a_list.pop(2) ## with index
del a_list[2] ## with index
a_list.remove(5) ## first occurence of 5
a_list.clear() ## returns an empty list
del a_list ## removes the list completely
```

Extend a list with another list

```
list_1 = [4, 2]
list_2 = [1, 3]
list_1.extend(list_2) ## [4, 2, 1, 3]
```

Reversing and sorting

```
list_1.reverse() ## [3, 1, 2, 4]
list_1.sort() ## [1, 2, 3, 4]
```

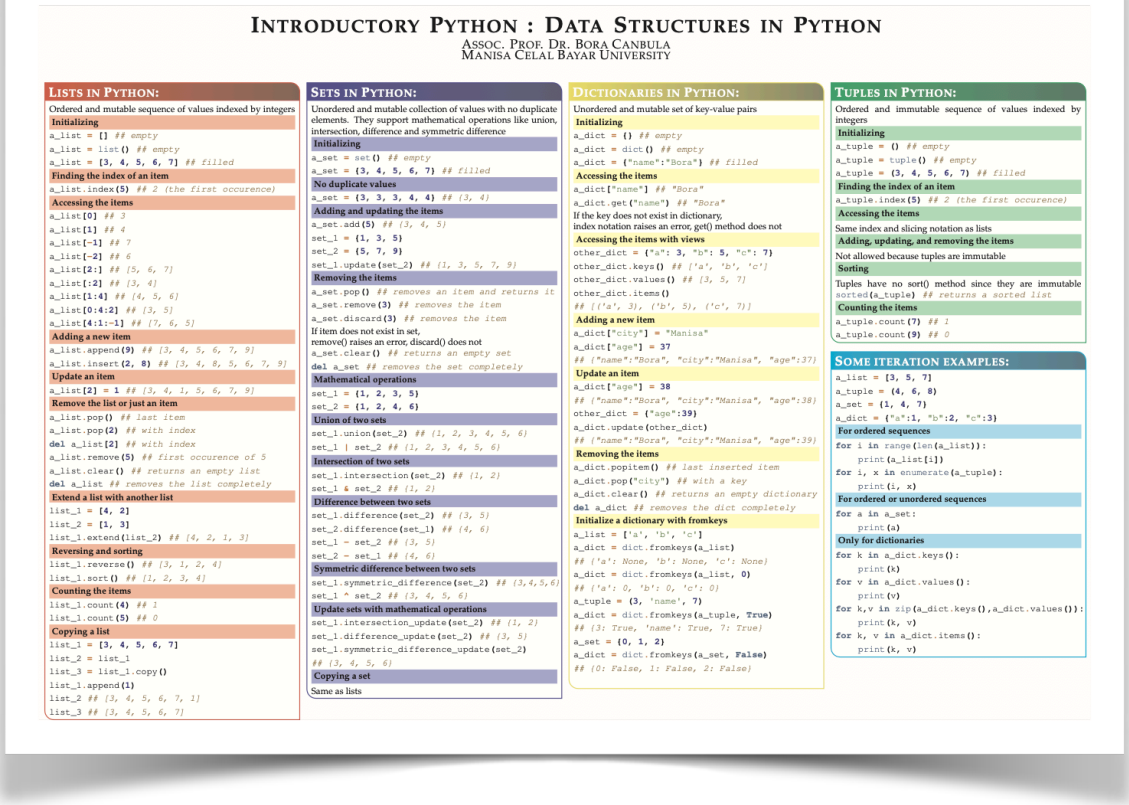
Counting the items

```
list_1.count(4) ## 1
list_1.count(5) ## 0
```

Copying a list

```
list_1 = [3, 4, 5, 6, 7]
list_2 = list_1
list_3 = list_1.copy()
list_1.append(1)
list_2 ## [3, 4, 5, 6, 7, 1]
list_3 ## [3, 4, 5, 6, 7]
```

Week03/IntroductoryPythonDataStructures.pdf



Introductory Python : Data Structures in Python

Assoc. Prof. Dr. Bora Canbula
Manisa Celal Bayar University

Lists in Python:

Ordered and mutable sequence of values indexed by integers

Initializing
```
a_list = [] ## empty
a_list = list() ## empty
a_list = [3, 4, 5, 6, 7] ## filled
```
Finding the index of an item
```
a_list.index(5) ## 2 (the first occurence)
```
Accessing the items
```
a_list[0] ## 3
a_list[1] ## 4
a_list[-1] ## 7
a_list[-2] ## 6
a_list[2:] ## [5, 6, 7]
a_list[:2] ## [3, 4]
a_list[1:4] ## [4, 5, 6]
a_list[0:4:2] ## [3, 5]
a_list[4:1:-1] ## [7, 6, 5]
```
Adding a new item
```
a_list.append(9) ## [3, 4, 5, 6, 7, 9]
a_list.insert(2, 8) ## [3, 4, 8, 5, 6, 7, 9]
```
Update an item
```
a_list[2] = 1 ## [3, 4, 1, 5, 6, 7, 9]
```
Remove the list or just an item
```
a_list.pop() ## last item
a_list.pop(2) ## with index
del a_list[2] ## with index
a_list.remove(5) ## first occurence of 5
a_list.clear() ## returns an empty list
del a_list ## removes the list completely
```
Extend a list with another list
```
list_1 = [4, 2]
list_2 = [1, 3]
list_1.extend(list_2) ## [4, 2, 1, 3]
```
Reversing and sorting
```
list_1.reverse() ## [3, 1, 2, 4]
list_1.sort() ## [1, 2, 3, 4]
```
Counting the items
```
list_1.count(4) ## 1
list_1.count(5) ## 0
```
Copying a list
```
list_1 = [3, 4, 5, 6, 7]
list_2 = list_1
list_3 = list_1.copy()
list_1.append(1)
list_2 ## [3, 4, 5, 6, 7, 1]
list_3 ## [3, 4, 5, 6, 7]
```

Sets in Python:

Unordered and mutable collection of values with no duplicate elements. They support mathematical operations like union, intersection, difference and symmetric difference

Initializing
```
a_set = set() ## empty
a_set = {3, 4, 5, 6, 7} ## filled
```
No duplicate values
```
a_set = {3, 3, 3, 4, 4} ## {3, 4}
```
Adding and updating the items
```
a_set.add(5) ## {3, 4, 5}
set_1 = {1, 3, 5}
set_2 = {5, 7, 9}
set_1.update(set_2) ## {1, 3, 5, 7, 9}
```
Removing the items
```
a_set.pop() ## removes an item and returns it
a_set.remove(3) ## removes the item
a_set.discard(3) ## removes the item
```
If item does not exist in set,
remove() raises an error, discard() does not
```
a_set.clear() ## returns an empty set
del a_set ## removes the set completely
```
Mathematical operations
```
set_1 = {1, 2, 3, 5}
set_2 = {1, 2, 4, 6}
```
Union of two sets
```
set_1.union(set_2) ## {1, 2, 3, 4, 5, 6}
set_1 | set_2 ## {1, 2, 3, 4, 5, 6}
```
Intersection of two sets
```
set_1.intersection(set_2) ## {1, 2}
set_1 & set_2 ## {1, 2}
```
Difference between two sets
```
set_1.difference(set_2) ## {3, 5}
set_2.difference(set_1) ## {4, 6}
set_1 - set_2 ## {3, 5}
set_2 - set_1 ## {4, 6}
```
Symmetric difference between two sets
```
set_1.symmetric_difference(set_2) ## {3,4,5,6}
set_1 ^ set_2 ## {3, 4, 5, 6}
```
Update sets with mathematical operations
```
set_1.intersection_update(set_2) ## {1, 2}
set_1.difference_update(set_2) ## {3, 5}
set_1.symmetric_difference_update(set_2)
## {3, 4, 5, 6}
```
Copying a set

Same as lists

Dictionaries in Python:

Unordered and mutable set of key-value pairs

Initializing
```
a_dict = {} ## empty
a_dict = dict() ## empty
a_dict = {"name":"Bora"} ## filled
```
Accessing the items
```
a_dict["name"] ## "Bora"
a_dict.get("name") ## "Bora"
```
If the key does not exist in dictionary,
index notation raises an error, get() method does not

Accessing the items with views
```
other_dict = {"a": 3, "b": 5, "c": 7}
other_dict.keys() ## ['a', 'b', 'c']
other_dict.values() ## [3, 5, 7]
other_dict.items()
## [('a', 3), ('b', 5), ('c', 7)]
```
Adding a new item
```
a_dict["city"] = "Manisa"
a_dict["age"] = 37
## {"name":"Bora", "city":"Manisa", "age":37}
```
Update an item
```
a_dict["age"] = 38
## {"name":"Bora", "city":"Manisa", "age":38}
other_dict = {"age":39}
a_dict.update(other_dict)
## {"name":"Bora", "city":"Manisa", "age":39}
```
Removing the items
```
a_dict.popitem() ## last inserted item
a_dict.pop("city") ## with a key
a_dict.clear() ## returns an empty dictionary
del a_dict ## removes the dict completely
```
Initialize a dictionary with fromkeys
```
a_list = ['a', 'b', 'c']
a_dict = dict.fromkeys(a_list)
## {'a': None, 'b': None, 'c': None}
a_dict = dict.fromkeys(a_list, 0)
## {'a': 0, 'b': 0, 'c': 0}
a_tuple = (3, 'name', 7)
a_dict = dict.fromkeys(a_tuple, True)
## {3: True, 'name': True, 7: True}
a_set = {0, 1, 2}
a_dict = dict.fromkeys(a_set, False)
## {0: False, 1: False, 2: False}
```

Tuples in Python:

Ordered and immutable sequence of values indexed by integers

Initializing
```
a_tuple = () ## empty
a_tuple = tuple() ## empty
a_tuple = (3, 4, 5, 6, 7) ## filled
```
Finding the index of an item
```
a_tuple.index(5) ## 2 (the first occurence)
```
Accessing the items

Same index and slicing notation as lists

Adding, updating, and removing the items

Not allowed because tuples are immutable

Sorting

Tuples have no sort() method since they are immutable
```
sorted(a_tuple) ## returns a sorted list
```
Counting the items
```
a_tuple.count(7) ## 1
a_tuple.count(9) ## 0
```

Some iteration examples:

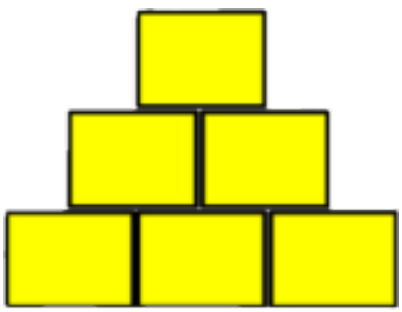
```
a_list = [3, 5, 7]
a_tuple = (4, 6, 8)
a_set = {1, 4, 7}
a_dict = {"a":1, "b":2, "c":3}
```
For ordered sequences
```
for i in range(len(a_list)):
    print(a_list[i])
for i, x in enumerate(a_tuple):
    print(i, x)
```
For ordered or unordered sequences
```
for a in a_set:
    print(a)
```
Only for dictionaries
```
for k in a_dict.keys():
    print(k)
for v in a_dict.values():
    print(v)
for k,v in zip(a_dict.keys(),a_dict.values()):
    print(k, v)
for k, v in a_dict.items():
    print(k, v)
```



https://forms.cloud.microsoft/r/... in-class quiz



Week03/pyramid_first_last.py



```python
def calculate_pyramid_height(number_of_blocks):



    return height
```



Week03/sequences_first_last.py

```python
def remove_duplicates(seq: list) -> list:
    """
    This function removes duplicates from a list.
    """
    return ...

def list_counts(seq: list) -> dict:
    """
    This function counts the number of
    occurrences of each item in a list.
    """
    return ...

def reverse_dict(d: dict) -> dict:
    """
    This function reverses the keys
    and values of a dictionary.
    """
    return ...
```

Exception Handling

Exception handling is a mechanism in Python to handle runtime errors gracefully without crashing the program.

```
try:
    # Code that may raise an exception
except Exception as e:
    # Code to handle the exception
else:
    # Code to execute if no exception is raised
finally:
    # Code that will be executed regardless of the result
```

Handling an Exception

```
try:
    1 / 0
except ZeroDivisionError:
    print("You cannot divide by zero.")
```

Handling All Exceptions

```
try:
    number = int(input("Enter a number: "))
    result = 1 / number
except Exception as e:
    print("An error occurred:", e)
```

Using finally Block

```
try:
    file = open("example.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found.")
else:
    print("File read successfully.")
    print("Content:", content)
finally:
    file.close()
    print("File closed.")
```

Handling Multiple Exceptions

```
try:
    number = int(input("Enter a number: "))
    result = 1 / number
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("You cannot divide by zero.")
```

Using else Block

```
try:
    number = int(input("Enter a number: "))
    result = 1 / number
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("You cannot divide by zero.")
else:
    print("The result is:", result)
```

Raising Exceptions

```
def calculate_area(radius):
    if radius < 0:
        raise ValueError("No negative radius!")
    return 3.14 * radius ** 2

try:
    area = calculate_area(-5)
except ValueError as e:
    print("An error occurred:", e)
```

Can we create custom exceptions?

Functions

Functions are defined by using def keyword, name, and the parenthesized list of formats parameters.

Naming Convention from PEP8

> Function names should be lowercase, with words separated by underscores as necessary to improve the readability.

Basic Function Definition

```
def function_name():
    pass
```

Input and Output Arguments

```
def fn(arg1, arg2):
    return arg1 + arg2
```

Default Values for Arguments

```
def fn(arg1 = 0, arg2 = 0):
    return arg1 + arg2
```

Type Hints and Default Values for Arguments

```
def fn(arg1: int = 0, arg2: int = 0) -> int:
    return arg1 + arg2
```

PEP 3107

Multiple Type Hints for Arguments (> Python 3.10)

```
def fn(arg1: int|float, arg2: int|float) -> (float, float):
    return arg1 * arg2, arg1 / arg2
```

> Python 3.10

Lambda Functions

```
fn = lambda arg1, arg2: arg1 + arg2
```

Function Docstrings

```
def fn(arg1 = 0, arg2 = 0):
    """This function sums two number."""
    return arg1 + arg2
```

PEP 257

Docstrings PEP 257

A docstring is a string literal that occurs as the first statement in a module, function, class or method definition. Such a docstring becomes the __doc__ special attribute of that object.

One-line Docstrings

```
def fn(arg1 = 0, arg2 = 0):
    """This function sums two number."""
    return arg1 + arg2
```

Multi-line Docstrings

```
def fn(arg1 = 0, arg2 = 0):
    """This function sums two number.

    Keyword arguments:
    arg1 -- first number (default 0)
    arg2 -- second number (default 0)
    Return: the sum of arg1 and arg2
    """
    return arg1 + arg2
```

Docutils and Sphinx are tools to automatically create documentations

reST (reStructuredText)

```
def fn(arg1 = 0, arg2 = 0):
    """
    This function sums two number.

    :param arg1: First number
    :param arg2: Second number
    :return: Sum of two numbers
    """
    return arg1 + arg2
```

Some other formats are Epytext (javadoc), Numpydoc, etc.

Google

```
def fn(arg1 = 0, arg2 = 0):
    """
    This function sums two number.

    Args:
        arg1 (int): First number
        arg2 (int): Second number

    Returns:
        int: Sum of two numbers
    """
    return arg1 + arg2
```

Parameter Kinds PEP 362

Kind describes how argument values are bound to the parameter. The kind can be fixed in the signature of the function.

Positional-or-Keyword (Standard Binding)

```
def fn(arg1 = 0, arg2 = 0):
    return arg1 + arg2

fn(), fn(3), fn(3, 5), fn(arg1=3), fn(arg2=5), fn(arg1=3, arg2=5)
```

Positional-or-Keyword and Keyword-Only

```
def fn(arg1 = 0, arg2 = 0, *, arg3 = 1):
    return (arg1 + arg2) * arg3

fn(), fn(3), fn(3, 5), fn(3, 5, 2), fn(arg1=3), fn(arg2=5), fn(arg1=3, arg2=5),
fn(3, 5, arg3=2), fn(arg1=3, arg2=5, arg3=2), fn(arg3=2, arg1=3, arg2=5)
```

Positional-Only and Positional-or-Keyword and Keyword-Only

```
def fn(arg1=0, arg2=0, /, arg3=1, arg4=1, *, arg5=1, arg6=1):
    return (arg1 + arg2) * arg3 / arg4 * arg5**arg6

fn(), fn(3), fn(3, 5), fn(3, 5, 2), fn(3, 5, 2, 4), fn(3, 5, 2, 4, 7),
fn(3, 5, arg3=2, arg4=4), fn(arg1=3, arg2=5, arg3=2, arg4=4),
fn(3, 5, arg3=2, arg4=4, arg5=7, arg6=8), fn(3, 5, 2, 4, arg5=7, arg6=8),
fn(arg1=3, arg2=5, arg3=2, arg4=4, arg5=7, arg6=8)
```

PEP 457

*args and **kwargs

```
def fn(*args, **kwargs):
    print(args) # a tuple of positional arguments
    print(kwargs) # a dictionary of keyword arguments

fn(), fn(3), fn(3, 5), fn(x=3, y=5), fn(3, 5, x=2, y=4),
fn(3, 5, x=2, y=4, 1, 2)
```

Function Attributes PEP 232

Functions already have a number of attributes such as __doc__, __annotations__, __defaults__, etc. Like everything in Python, functions are also objects, therefore, user can add a dictionary as attributes by using get / set methods to __dict__.

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions

A	**E**	**L**	**R**
abs()	enumerate()	len()	range()
aiter()	eval()	list()	repr()
all()	exec()	locals()	reversed()
anext()			round()
any()	**F**	**M**	
ascii()	filter()	map()	**S**
	float()	max()	set()
B	format()	memoryview()	setattr()
bin()	frozenset()	min()	slice()
bool()			sorted()
breakpoint()	**G**	**N**	staticmethod()
bytearray()	getattr()	next()	str()
bytes()	globals()		sum()
		O	super()
C	**H**	object()	
callable()	hasattr()	oct()	**T**
chr()	hash()	open()	tuple()
classmethod()	help()	ord()	type()
compile()	hex()		
complex()		**P**	**V**
	I	pow()	vars()
D	id()	print()	
delattr()	input()	property()	**Z**
dict()	int()		zip()
dir()	isinstance()		
divmod()	issubclass()		**_**
	iter()		__import__()

Function Attributes PEP 232

Functions already have a number of attributes such as __doc__, __annotations__, __defaults__, etc. Like everything in Python, functions are also objects, therefore, user can add a dictionary as attributes by using get / set methods to __dict__.

setattr(*object*, *name*, *value*)

This is the counterpart of `getattr()`. The arguments are an object, a string, and an arbitrary value. The string may name an existing attribute or a new attribute. The function assigns the value to the attribute, provided the object allows it. For example, `setattr(x, 'foobar', 123)` is equivalent to `x.foobar = 123`.

name need not be a Python identifier as defined in Identifiers and keywords unless the object chooses to enforce that, for example in a custom `__getattribute__()` or via `__slots__`. An attribute whose name is not an identifier will not be accessible using the dot notation, but is accessible through `getattr()` etc..

getattr(*object*, *name*)
getattr(*object*, *name*, *default*)

Return the value of the named attribute of *object*. *name* must be a string. If the string is the name of one of the object's attributes, the result is the value of that attribute. For example, `getattr(x, 'foobar')` is equivalent to `x.foobar`. If the named attribute does not exist, *default* is returned if provided, otherwise `AttributeError` is raised. *name* need not be a Python identifier (see `setattr()`).

hasattr(*object*, *name*)

The arguments are an object and a string. The result is `True` if the string is the name of one of the object's attributes, `False` if not. (This is implemented by calling `getattr(object, name)` and seeing whether it raises an `AttributeError` or not.)

delattr(*object*, *name*)

This is a relative of `setattr()`. The arguments are an object and a string. The string must be the name of one of the object's attributes. The function deletes the named attribute, provided the object allows it. For example, `delattr(x, 'foobar')` is equivalent to `del x.foobar`. *name* need not be a Python identifier (see `setattr()`).

Nested Scopes PEP 227

Function objects can have methods. These methods can be used as inner functions and can be useful for encapsulation.

```
def parent_function():
    def nested_function():
        print("Nested function")
    print("Parent function")
    parent_function.nested_function = nested_function
```



```
parent_function()
parent_function.nested_function()
```

Getter and Setter Methods

```
def point(x, y):
    def set_x(new_x):
        nonlocal x
        x = new_x
    def set_y(new_y):
        nonlocal y
        y = new_y
    def get():
        return x, y
    point.set_x = set_x
    point.set_y = set_y
    point.get = get
    return point
```

Decorators

Decorators take a function as argument and returns a function. They are used to extend the behavior of the wrapper function, without modifying it. So they are very useful for dealing with code legacy.

Traditional Way

```
def my_decorator(fn):
    def _my_decorator():
        print("Before function")
        fn()
        print("After function")
    return _my_decorator

def my_decorated_function():
    print("Function")

my_decorated_function = \
    my_decorator(my_decorated_function)
```

Pythonic Way

```
def d1(fn):
    def _d1():
        print("Before d1")
        fn()
        print("After d1")
    return _d1

@d1
def f1():
    print("Function")
```

Decorators with Arguments

```
def decorator(func):
    def _decorator(*args, **kwargs):
        print("I am decorator")
        print(args)
        print(kwargs)
        func(*args, **kwargs)
    return _decorator

@decorator
def decorated_func_w_args(x):
    print(f"x = {x}")

@decorator
def decorated_triple_print(
    x=None, y=None, z=None):
    x_str = f"x = {x} " \
        if x is not None else f""
    y_str = f"y = {y} " \
        if y is not None else f""
    z_str = f"z = {z} " \
        if z is not None else f""
    print(x_str + y_str + z_str)
```

Decorator Chain

```
def d1(func):
    def _d1(*args, **kwargs):
        print(f"d1 here for \
              {func.__name__}")
        func(*args, **kwargs)
    return _d1

def d2(func):
    def _d2(*args, **kwargs):
        print(f"d2 here for \
              {func.__name__}")
        func(*args, **kwargs)
    return _d2

@d1
@d2
def fd(x):
    print(f"f says {x}")
```



https://forms.cloud.microsoft/r/

in-class quiz

Week04/functions_firstname_lastname.py

custom_power

- A lambda function
- Two parameters (x and e)
- x is positional-only
- e is positional-or-keyword
- x has the default value 0
- e has the default value 1
- Returns x**e

custom_equation

- A function returns float
- Five integer parameters (x, y, a, b, c)
- x is positional-only with default value 0
- y is positional-only with default value 0
- a is positional-or-keyword with default value 1
- b is positional-or-keyword with default value 1
- c is keyword-only with default value 1
- Function signature must include all annotations
- Docstring must be in reST format.
- Returns (x**a + y**b) / c

fn_w_counter

- A function returns a tuple of an int and a dictionary
- Function must count the number of calls with caller information
- Returning integer is the total number of calls
- Returning dictionary with string keys and integer values includes the caller (_ _name_ _) as key, the number of call coming from this caller as value.

Examples

```
custom_power(2) == 2
custom_power(2, 3) == 8
custom_power(2, e=2) == 4
custom_equation(2, 3) == 5.0
custom_equation(2, 3, 2) == 7.0
custom_equation(2, 3, 2, 3) == 31.0
custom_equation(3, 5, a=2, b=3, c=4) == 33.5
custom_equation(3, 5, 2, b=3, c=4) == 33.5
custom_equation(3, 5, 2, 3, c=4) == 33.5
for i in range(10):
    fn_w_counter()
fn_w_counter() == (11, {'__main__': 11})
```

Week04/decorators_firstname_lastname.py

performance

- A decorator which measures the performance of functions and also saves some statistics.
- Has three attributes: counter, total_time, total_mem
- Attribute counter stores the number of times that the decorator has been called.
- Attribute total_time stores the number of total time that the functions took.
- Attribute total_mem stores the total memory in bytes that the functions consumed.